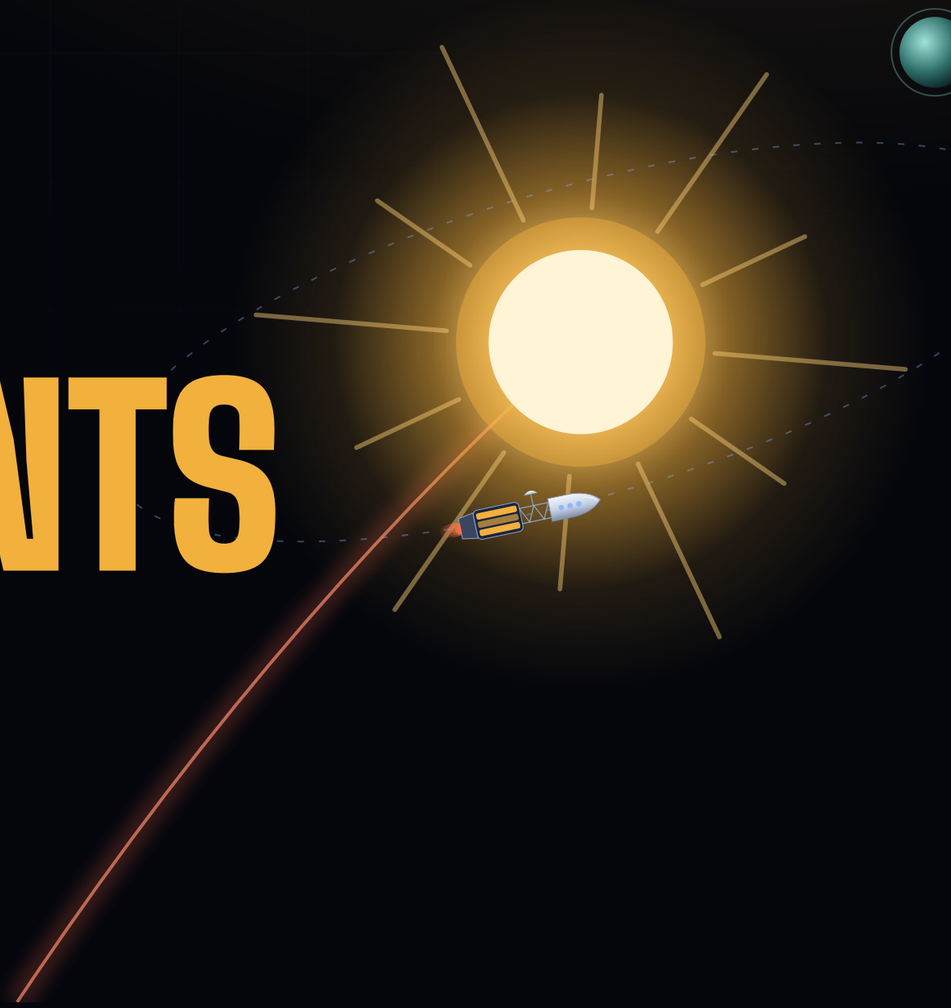
MLOpsDay · 2026

HOW I LEARNED TO STOP WORRYING AND SHIP AI AGENTS TO PRODUCTION

Salvatore Visaggi AI Engineer @ Agilelab



MISSION PLAN

FROM SOL TO TAU CETI IN SIX BURNS



01 · BRIEF

WAKE UP

Harness and runtime

02 · SHORT

MISSION A

Insurance claims: a workflow, on purpose

03 · MAIN

MISSION B

Talking to data: the real agent

04 · FOCUS

FIRST CONTACT

UX: show something useful, fast

05 · FOCUS

DEEP SPACE

Traced, reproducible, judged

06 · HOME

RETURN TRIP

What we'd do differently, the checklist

Both missions are composites of real projects. Names and data are illustrative; the lessons are not.

BEFORE WE LAUNCH

WHAT WORRIES YOU MOST ABOUT SHIPPING AN AGENT?

A
QUALITY
Wrong answers, said with confidence

B
COST
The invoice after the demo

C
SECURITY
Data leaks, prompt injection, permissions

D
LATENCY
Users staring at a spinner

E
ADOPTION
It works, and nobody uses it

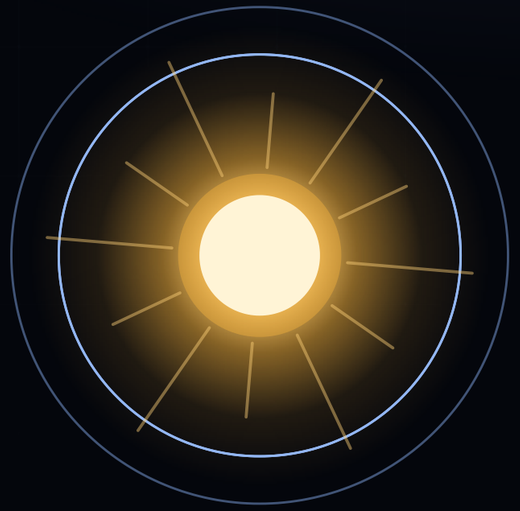


♪ Keep one in mind. Question? ♫

01
WAKE UP

The basics, fast: what an agent needs besides a model

> LOG · Subject is awake. Knows a great deal of science. Remembers nothing about the mission.



THE PREMISE

THE MODEL IS THE CORE. EVERYTHING ELSE IS THE SHIP.



Every piece around the model is a bet on what it can't do yet. Check them again at every model upgrade.

WORKFLOW ↔ AGENT

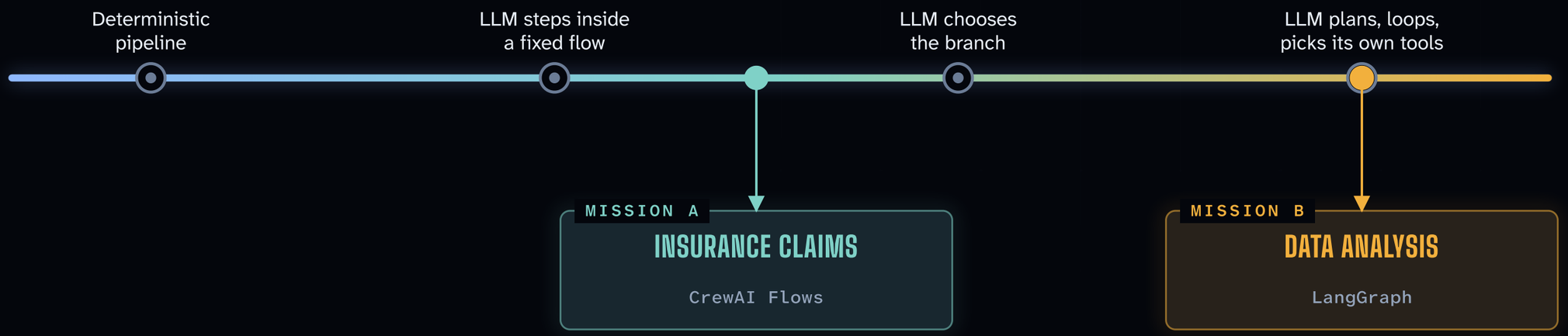
FIXED TRAJECTORY OR FREE FLIGHT?

FIXED TRAJECTORY

CODE DECIDES THE NEXT STEP

FREE FLIGHT

THE MODEL DECIDES THE NEXT STEP



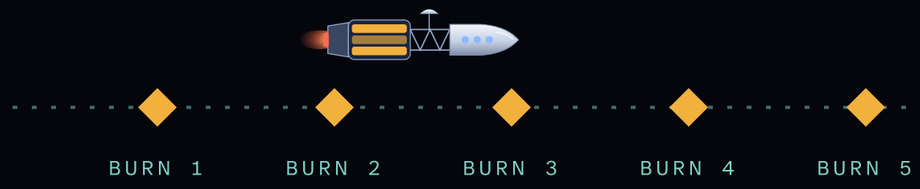
The design question is who decides the next step: your code or the model. Pick the left-most point that still solves the problem.



♪ Crew poll: a data analysis agent. Which point, question? ♫

02

MISSION A: THE CLAIMS RUN

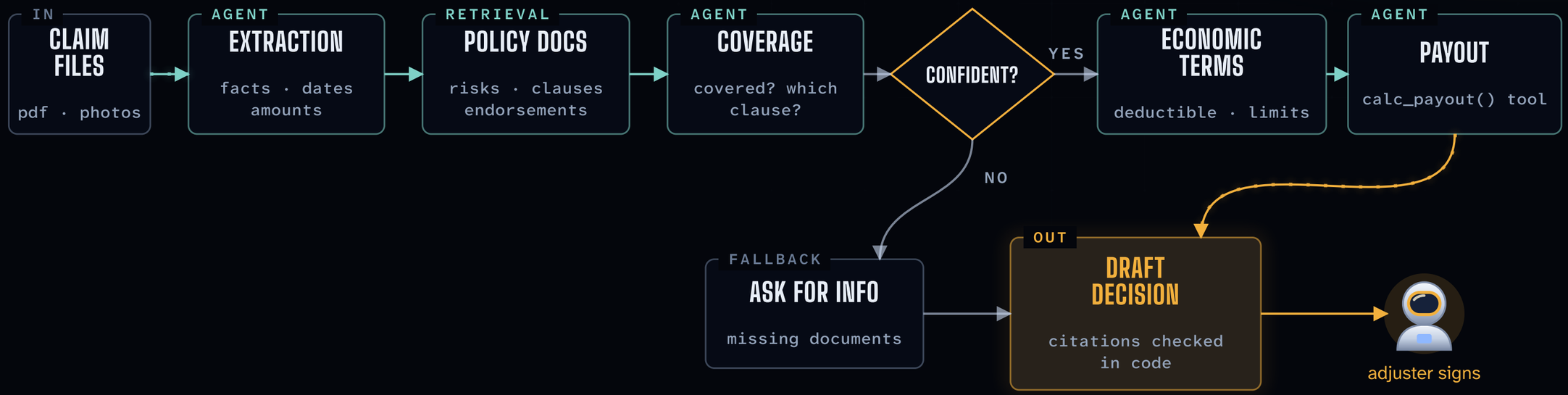


Insurance assistant. A workflow, on purpose. Short, as a contrast.

> LOG · Trajectory computed before launch. Every burn is on the checklist.

MISSION A · INSURANCE

NOT A CHATBOT. A FLOW.



IN · invoices, prescriptions

OUT · a decision draft

EACH AGENT · ITS TOOLS, ITS DOCUMENTS

CrewAI · MLflow · Datadog (OTel)

MISSION A · PAYOUT

ARITHMETIC IN CODE, EVERY NUMBER CITED

Health policy, medical expenses. The MRI and the specialist visit are covered as outpatient care [1]. Invoices €1,250, 20% co-insurance, minimum €100 [2]. The annual limit has €800 left [3]. Proposed payout: **€800**.

1 · Policy §4.2 · p.12 2 · §4.2.b · p.12 3 · §7.1 · p.21

```python
def calc_payout(amount: Decimal, terms: Terms) -> Payout:
    """Apply the cover terms to one claim.
    Terms arrive already resolved:
    only the clauses that apply."""
    share = max(amount * terms.coinsurance, terms.min_share)
    due = max(amount - share, ZERO)
    paid = min(due, terms.limit_left)
    return Payout(paid, cites=terms.clauses)
```

FOUR STEPS, TWO KINDS OF WORKER

- **Agent** · extracts the terms from the policy and its endorsements, each with its validity dates.
- **Code** · decides which clause wins and drops the ones that don't apply.
- **Agent** · reasons on the claim and calls `calc_payout` with the terms that apply.
- **Code** · checks every citation: the span must exist in the document, or the step retries.

The LLM reads and cites. Code computes and checks.

MISSION A · TRUST OVER TIME

WHAT THE EVAL LOOP CAUGHT

CAUGHT · 01 · JUDGE + HUMAN REVIEW

AMBIGUOUS RULES

Some coverage and payout rules were vague in the policy documents. Adjusters shared the real ones informally.

Fix · make them explicit with the business. New policies get a gap analysis first.

Fewer
agent errors, with the rules written down

CAUGHT · 02 · GOLDEN SET

WRONG ARTICLES RETRIEVED

The right decision needs the right guarantee articles, and retrieval missed some.

Fix · the agent grades what it retrieves; retrieval is evaluated on its own.

Higher
share of articles retrieved correctly

CAUGHT · 03 · ADJUSTER FEEDBACK

TOO MANY ESCALATIONS

Early versions were rigid: low confidence, so a human almost every time.

Fix · clearer rules, better prompts and tools, business rules in code.

Fewer
escalations to a human, release after release

Every finding became a golden case. A regression can't sneak back in.

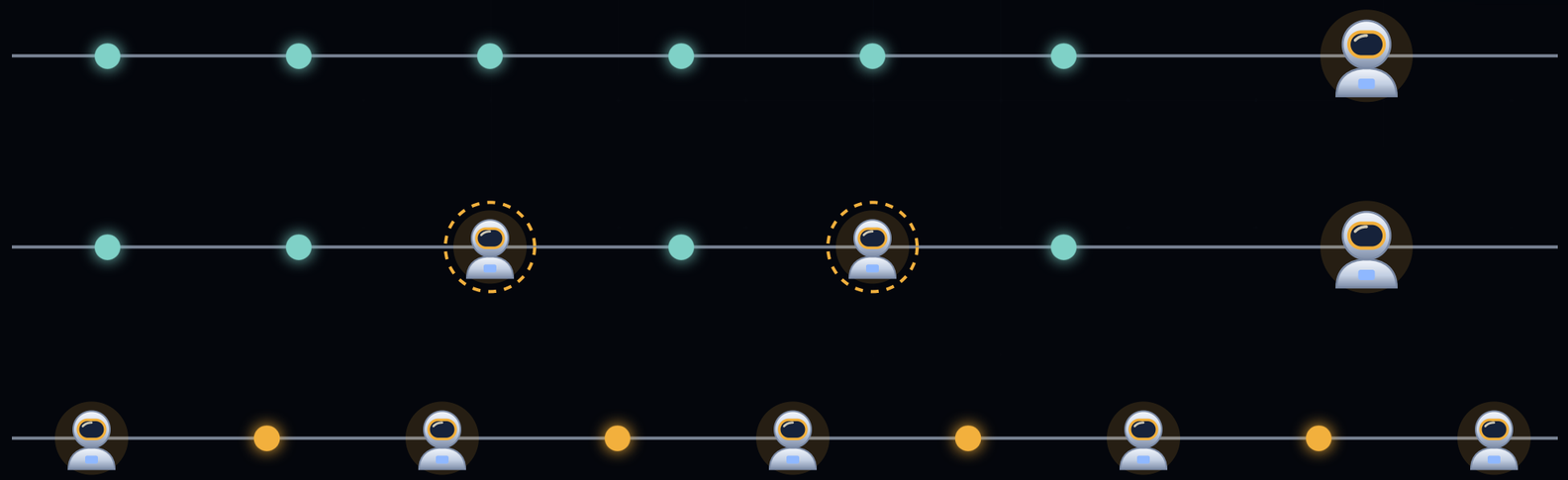
MISSION A · HUMAN IN THE LOOP

PUT THE HUMAN WHERE THE UNCERTAINTY IS

INSURANCE · PILOT
FINAL DECISION ONLY

INSURANCE · NOW
IN THE FLOW, ON CONDITIONS

DATA AGENT
EVERY TURN, FROM DAY ZERO

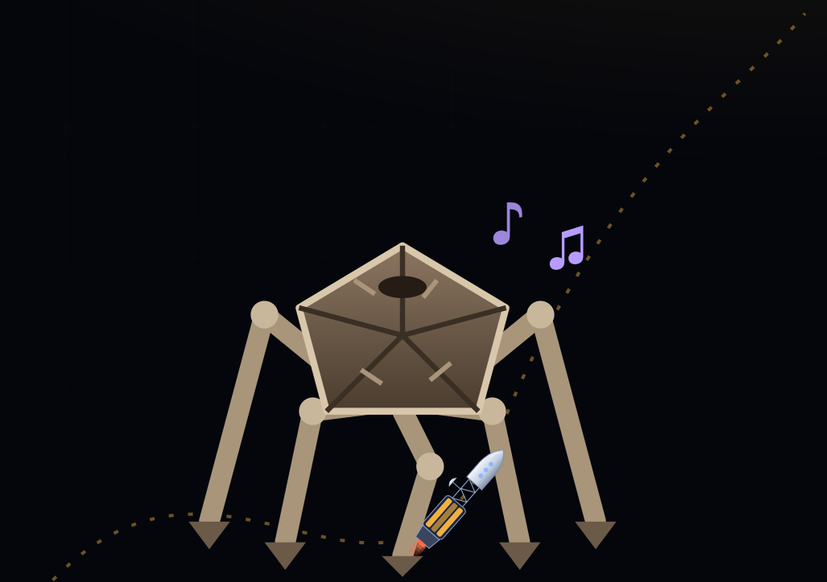


Pilots put the human at the end because it's easy. Production puts the human where the agent is unsure, and a conversational agent has one on every turn.

03

MISSION B: TALKING TO DATA

Conversational data analysis agent. The main event.



> LOG · No flight plan survives contact with the user. Improvise, but keep the logs.

CREW POLL · HANDS UP

HOW MANY CORRECT ANSWERS DOES THIS QUESTION HAVE?

How much did we sell last year, by region?



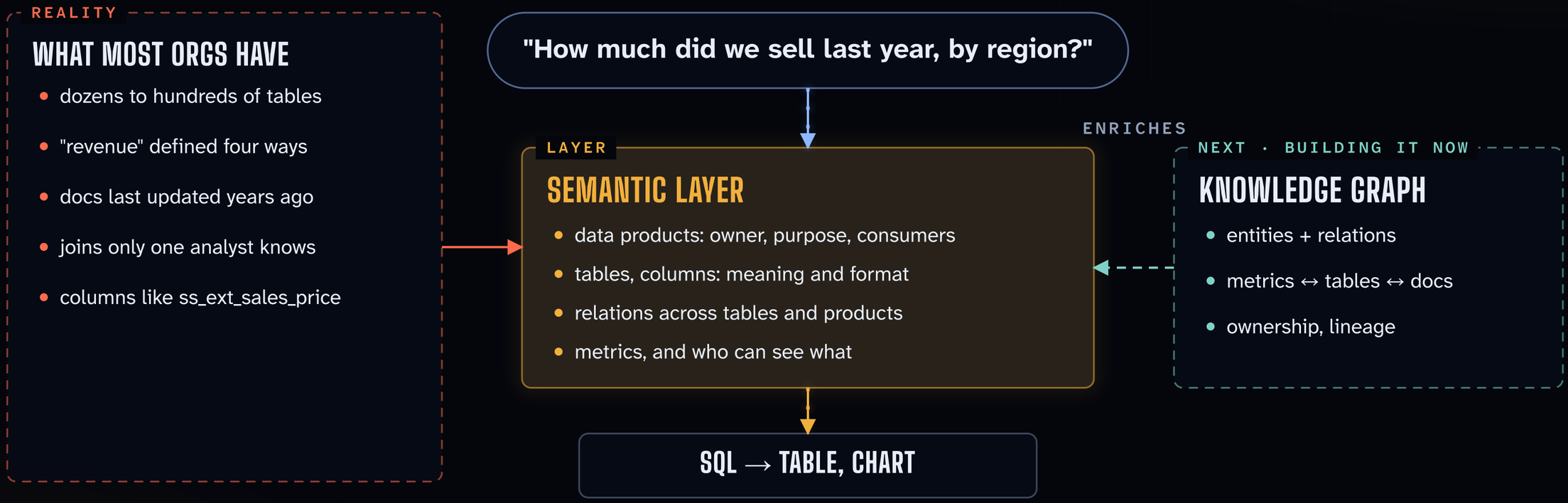
♪ One? Two? Ten? Question? ♫

SELL	gross · net of returns
LAST YEAR	calendar 2025 · fiscal year, April to March
SOLD ON	order date · ship date · invoice date
REGION OF	the customer · the store

2 × 2 × 3 × 2 = up to 24 correct-looking answers.
Only one is what the user meant.

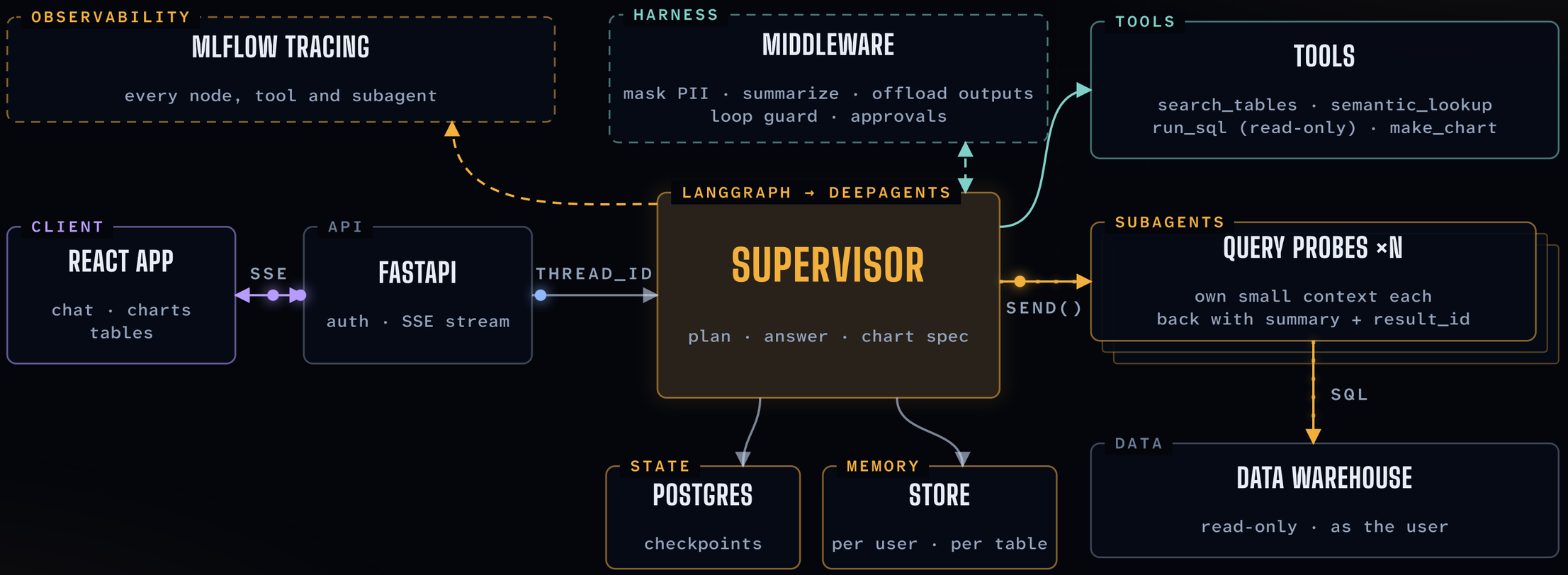
MISSION B · THE REAL BLOCKER

FIRST, BUILD A SHARED LANGUAGE



MISSION B · ARCHITECTURE

A SUPERVISOR AND ITS PROBES



MISSION B · HUMAN IN THE LOOP

A CREWMATE FROM DAY ZERO




```python
from langgraph.types import interrupt

def confirm_plan(state: State) -> dict[str, Plan]:
    """Stop until the user confirms or edits the plan."""
    plan: Plan = state["plan"]  # steps + assumptions
    edited: Plan | None = interrupt({"plan": plan})
    return {"plan": edited or plan}
```

BONUS

EVERY CORRECTION IS A LABEL

A plan the user fixes is a question paired with its right interpretation. It goes straight into the golden set.

MISSION B · CURATED CONTEXT

WHAT THE MODEL NEVER SEES

BY USER

NOT YOURS, NOT IN THE CATALOG

The semantic layer exposes only what this user may see. The agent can't ask for a table it doesn't know exists.

```
semantic_lookup("margin")
# → not found for this user
```

BY STEP

NOT NEEDED YET, NOT LOADED

Semantic models live as files. The agent lists them, opens what the question needs, and nothing else.

```
$ ls semantic/sales
net_revenue.yaml  returns.yaml
$ read semantic/sales/net_revenue.yaml
```

BY SENSITIVITY

NOT SAFE, MASKED

Middleware masks PII in every tool result, before the model reads it.

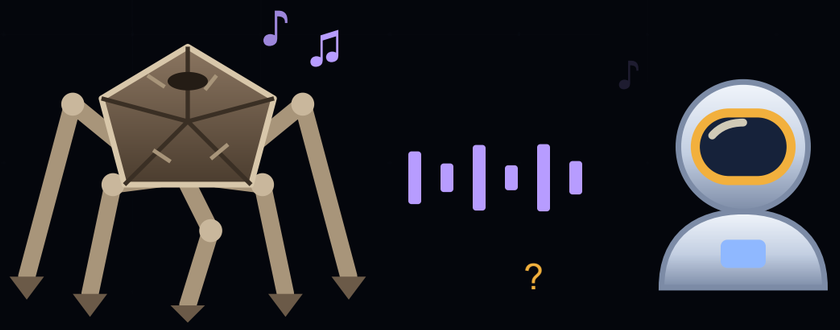
```
mario.rossi@example.com → <EMAIL_1>
IT60X054281110… → <IBAN_1>
```

Context is curated in code: who's asking, what this step needs, what's safe to show.

04

FIRST CONTACT

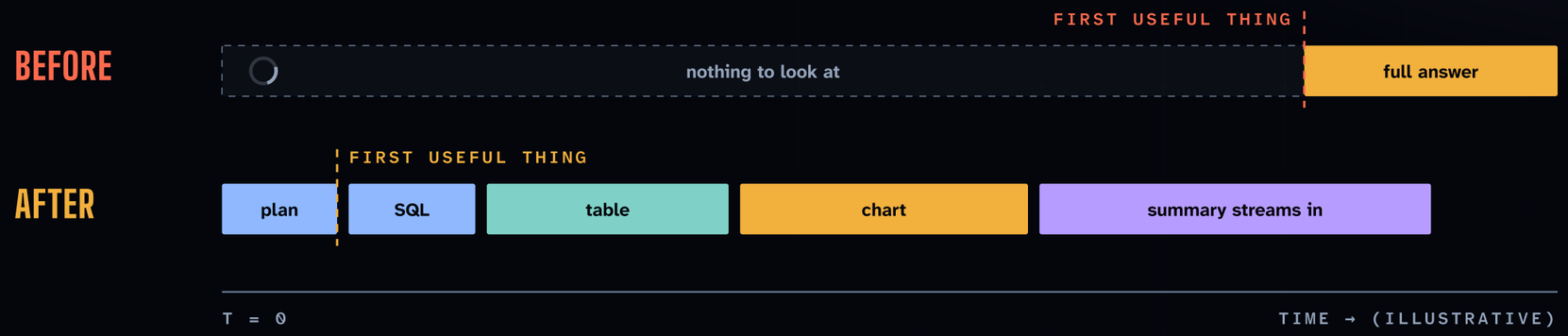
For a conversational agent, the user experience is the whole mission.



> LOG · Contact established. It takes more than chords and a spreadsheet to understand each other.

UX · LATENCY

LATENCY IS UX. SHOW SOMETHING USEFUL, FAST.



PATTERN · 01

STREAM THE STEPS

SSE events for plan, SQL, rows, chart. Waiting feels like progress.

PATTERN · 02

TABLE BEFORE CHART

Numbers first, pixels second, words last.

PATTERN · 03

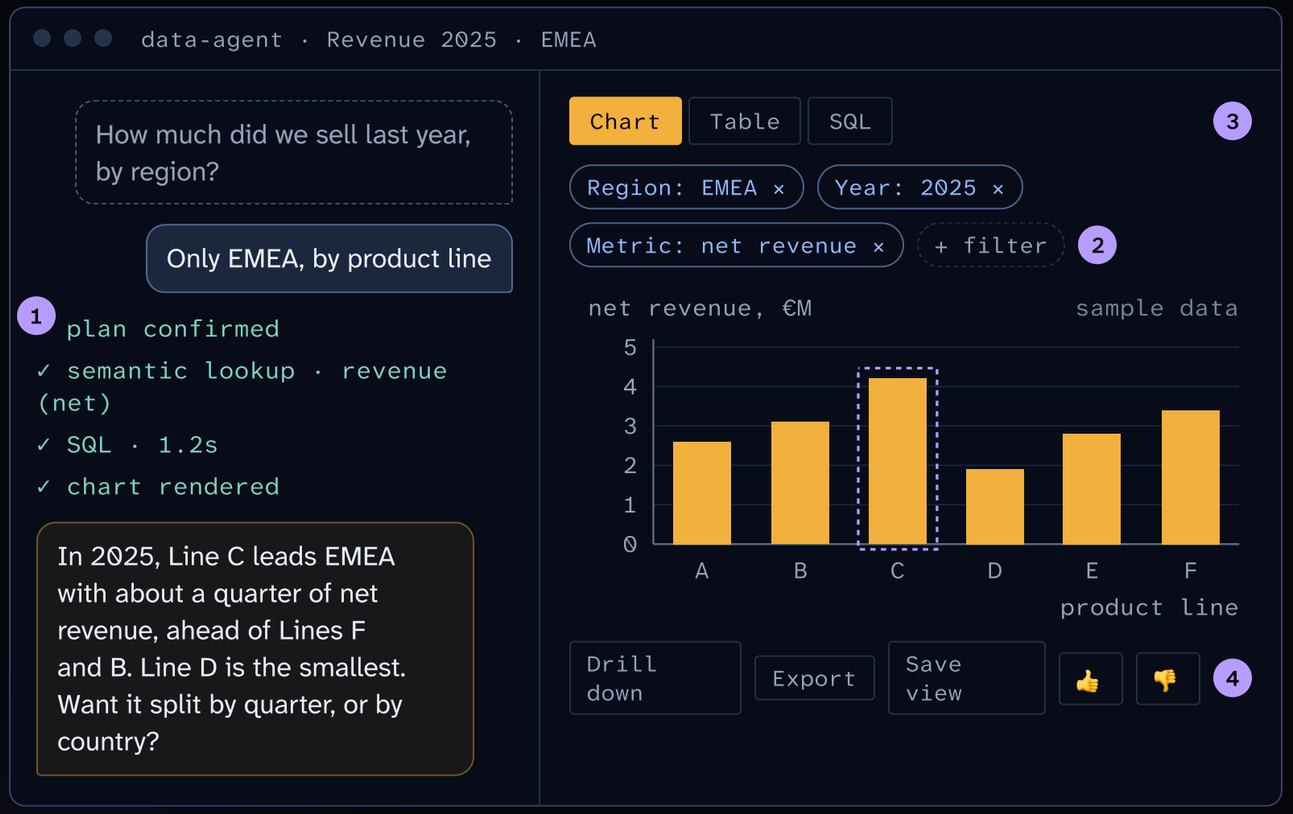
LET THEM CANCEL

A stop button beats a two-minute wrong query.

Time to first useful thing: from seconds staring at a spinner to almost instant.

UX · INTERACTION

CHARTS YOU CAN TOUCH



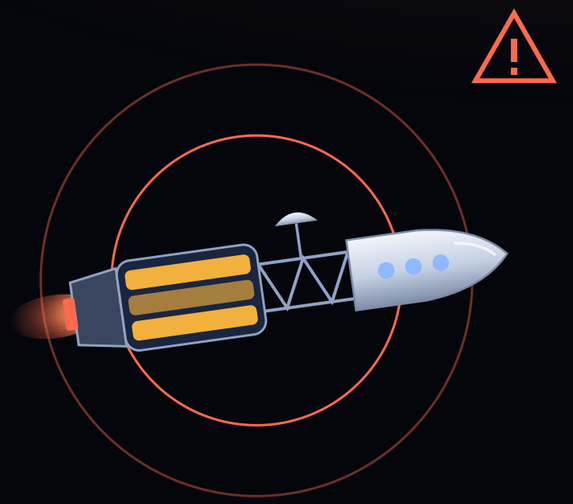
1. Live steps, so waiting feels like progress
2. Chips re-run the query on the result_id, no LLM: faster, cheaper, deterministic
3. Show the work: chart, table and SQL of the same result, one click apart
4. Feedback right where the answer is

05

DEEP SPACE

Production: traced, reproducible and judged, months after launch.

> LOG · Everything works on launch day. The trouble starts in month three.



PRODUCTION · DEBUGGING WITH MLFLOW

THE AGENT DIDN'T CRASH. IT KEPT TRYING.



A simple question, minutes of waiting. The loop is buried in one probe. Illustrative spans.

WHAT THE MODEL SAW

`run_sql → "query failed"`

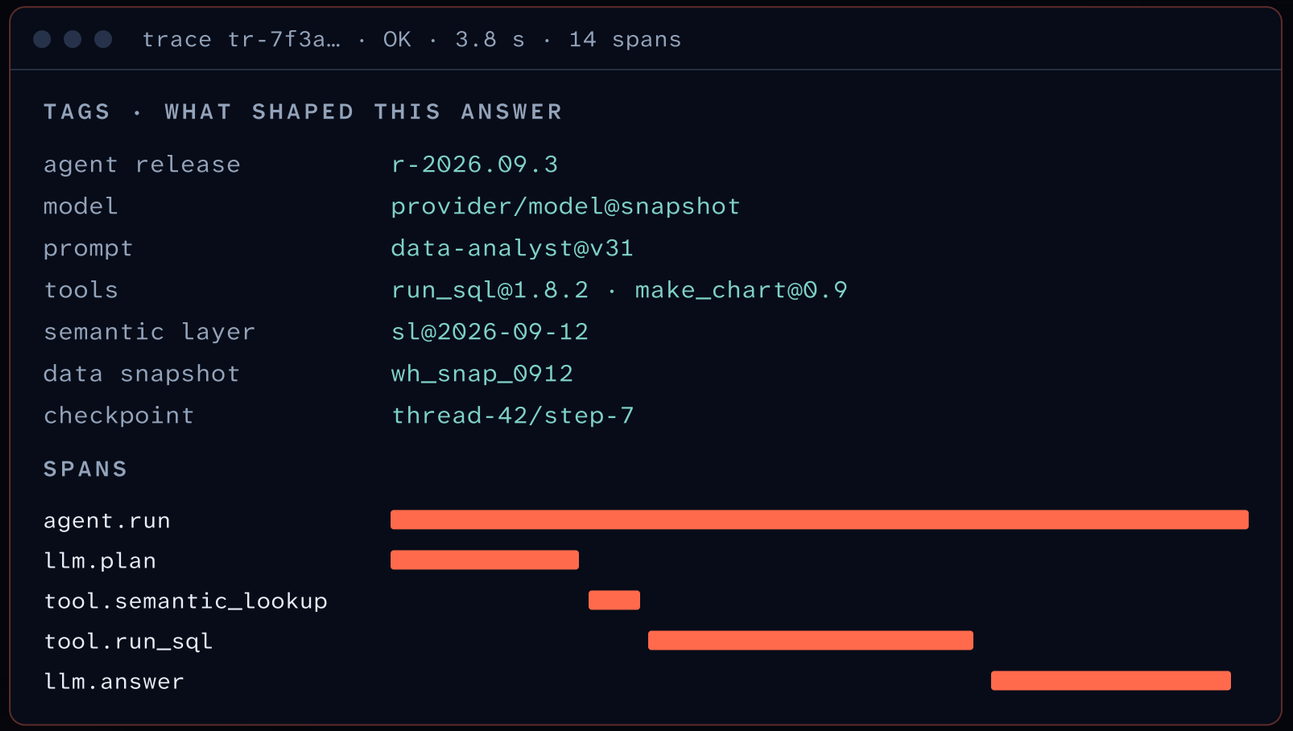
For a person, an annoying message. For the model, the whole prompt. So it guessed again.

FIXED OUTSIDE THE MODEL

- **Tool.** Returns the warehouse error: wrong column, wrong join
- **Middleware.** Blocks the same call repeated, and says why
- **Limit.** Two steps before the step limit, the agent stops, says what it tried, and asks

PRODUCTION · REPRODUCIBILITY

KEEP A FLIGHT RECORDER



Illustrative trace and values.

Same question, different answer is normal. Unexplainable is not.

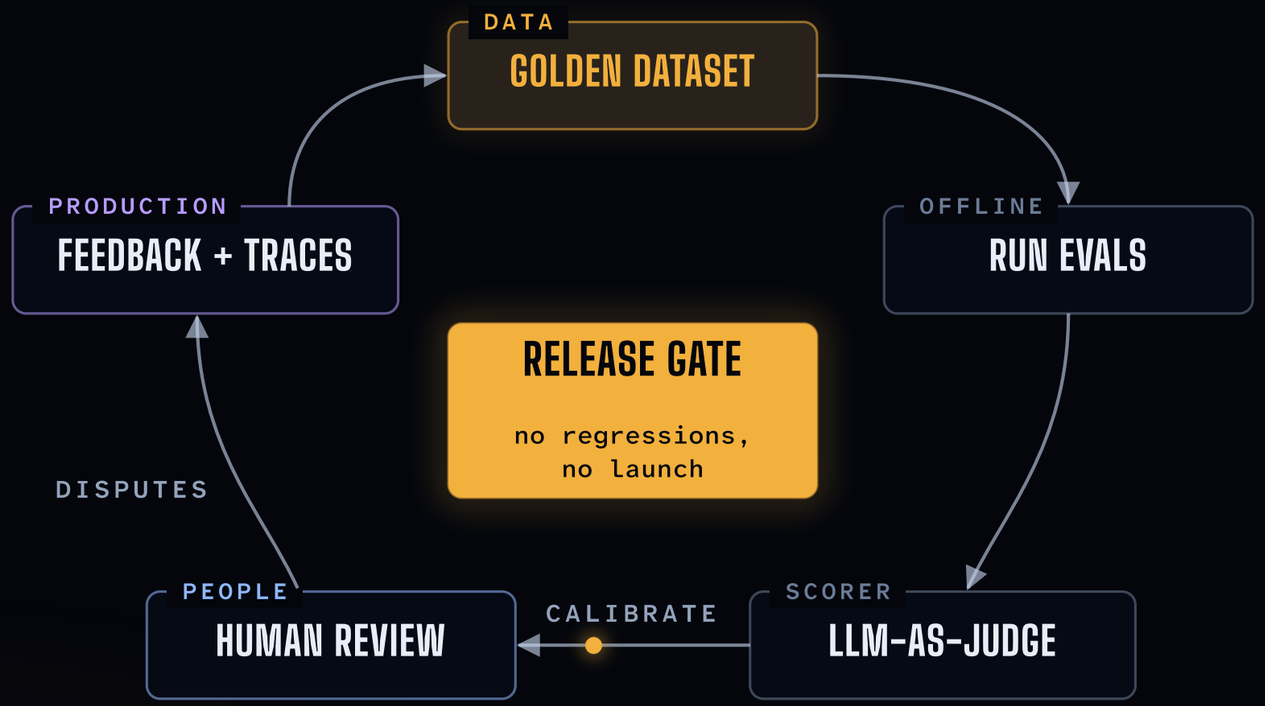
- Version everything: agent, model, prompt, tools, semantic layer, data
- Pin model snapshots, upgrade on purpose
- Freeze a data snapshot for evals
- Replay from a checkpoint to debug one step

WATCHED DAILY

cost / conversation | tool errors | latency / run | judge scores | time to first useful | escalations

PRODUCTION · EVALUATION

GOLDEN CASES COME FROM WHAT WENT WRONG



- **Where they come from.** Plan corrections, thumbs-down, edited SQL, human overrides: the hardest cases in real traffic
- **A** · before every release: amounts, outcome and confidence must not move; then judges check the approach and the cited documents
- **B** · same data snapshot: expected query and result, dataframe against dataframe

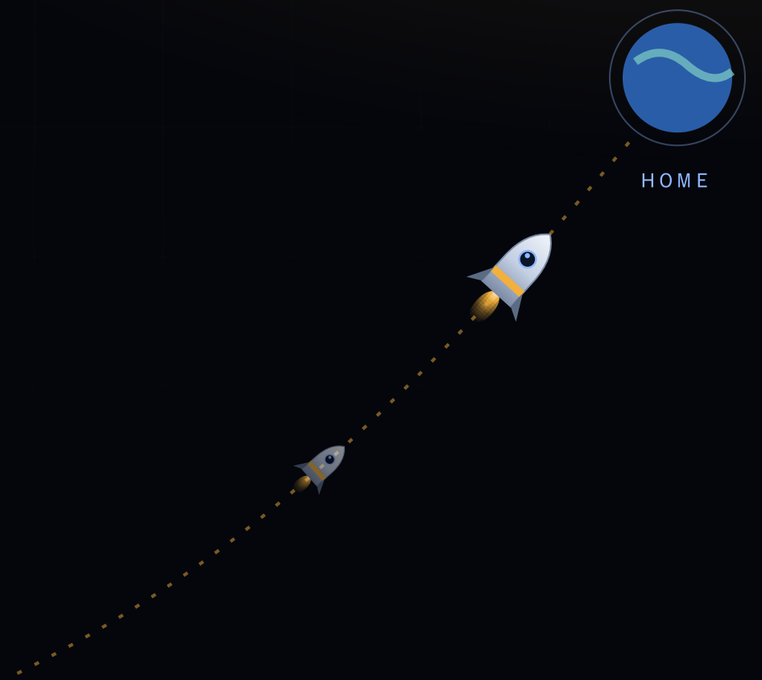
The worst cases became rules in the prompt. A golden set doesn't just measure: it tells you what to write.

06

RETURN TRIP

What we're bringing home: what we'd do differently, and a checklist for your next launch.

> LOG · Data secured. Heading home. Writing it all down for the next crew.



TAKEAWAY 1 · HONEST LOG

IF WE LAUNCHED AGAIN TOMORROW

LESSON · 01

SEMANTIC LAYER FIRST

The agent can't be smarter than your definitions. Agree on what "revenue" means, then build the agent on top.

LESSON · 02

A GOLDEN SET ON DAY ONE

Twenty real questions with checked answers beat a month of "looks good to me". It grows from there.

LESSON · 03

TRACING BEFORE THE FIRST USER

Not after the first incident. You can't fix a loop you can't see.

LESSON · 04

HUMANS IN THE FLOW FROM THE PILOT

A human only at the end is easy to add. Finding the uncertainty points early is cheaper.

TAKEAWAY 2 · GO / NO-GO

PRE-FLIGHT CHECKLIST

WILL IT BE RIGHT?

TRUSTED

- [ ] Tools as contracts, errors that say what next
- [ ] Guarantees in middleware, not prompts
- [ ] A golden set from day one
- [ ] A human where it's uncertain or irreversible

IS IT SAFE?

SECURE

- [ ] Acts as the user, read-only
- [ ] Context curated by user, step, sensitivity
- [ ] Documents and rows are data, never instructions
- [ ] An audit log of who asked and what ran

CAN WE AFFORD IT?

AFFORDABLE

- [ ] Cost per conversation, per release
- [ ] Token budgets in middleware
- [ ] A cache-aware context window
- [ ] References back to the model, not rows

CAN WE RUN IT?

OPERABLE

- [ ] Checkpoint, trace and version everything
- [ ] Evals gate every release
- [ ] Fallbacks, backoff, timeouts
- [ ] An owner, an on-call, SLOs

FINAL MESSAGE

NONE OF THE PROBLEMS IN THIS TALK WAS FIXED BY A BETTER MODEL

Every one was fixed by a better ship: the infrastructure around the LLM.

We didn't stop worrying. We started measuring.



MISSION DEBRIEF

QUESTIONS?

Feedback, slides: over to you.

Salvatore Visaggi · AI Engineer, Agilelab · linkedin.com/in/salvatore-visaggi

Slides and PDF
visalvo.github.io/
ship-ai-agents

Rate this talk
and send your
questions in the app